

Predictability-Aware Compression and Decompression Framework for Multichannel Time Series Data with Latent Seasonality

Ziqi Liu

Department of Computer Science
The University of Texas at Dallas
Dallas, TX, USA
ziqui.liu@utdallas.edu

Pei Zeng

Global AI & Platforms
LexisNexis
Raleigh, NC, USA
pei.zeng@lexisnexis.com

Yi Ding

Department of Computer Science
The University of Texas at Dallas
Dallas, TX, USA
yi.ding@utdallas.edu

Abstract

Real-world multichannel time series prediction faces growing demands for efficiency across edge and cloud environments, making channel compression a timely and essential problem. Motivated by success of Multiple-Input Multiple-Output (MIMO) methods in signal processing, we propose a predictability-aware compression-decompression framework to reduce runtime, decrease communication cost, and maintain prediction accuracy across diverse predictors. The core idea involves using a circular seasonal key matrix with orthogonality to capture underlying time series predictability during compression and to mitigate reconstruction errors during decompression by introducing more realistic data assumptions. Theoretical analyses show that the proposed framework is both time-efficient and accuracy-preserving under a large number of channels. Extensive experiments on six datasets across various predictors demonstrate that the proposed method achieves superior overall performance by jointly considering prediction accuracy and runtime, while maintaining strong compatibility with diverse predictors.

CCS Concepts

• **Computing methodologies** → Neural networks; • **Computer systems organization** → Sensor networks; • **Information systems** → Web services.

Keywords

Data compression, Deep learning, Time series prediction

ACM Reference Format:

Ziqi Liu, Pei Zeng, and Yi Ding. 2026. Predictability-Aware Compression and Decompression Framework for Multichannel Time Series Data with Latent Seasonality. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792488>

1 Introduction

Multichannel time series prediction is widely studied across various domains [15, 23, 31, 42] and has become increasingly critical in Web-scale applications, where time series data is transmitted

through Internet of Things (IoT) networks to Web platforms for large-scale access and decision support. In this context, a channel refers to a single signal stream or time-dependent variable—such as the reading from one sensor, a specific feature dimension, or a data source—recorded in parallel with other channels as part of a multichannel time series. These applications typically stream data from sensors to Web platforms, posing practical challenges for transmission and computation. For example, in large-scale IoT deployments, low-cost sensors rely on edge devices to aggregate local data and transfer it from Low Power Wide Area (LPWA) networks to IP link network [27]. In this case, as shown in Figure 1, multichannel data may originate from a single sensor [35] or from edge devices aggregating multiple sensors in areas without IP coverage [10, 47], both of which strain bandwidth and computation. Therefore, time series data compression for prediction task is a timely and valuable research direction.

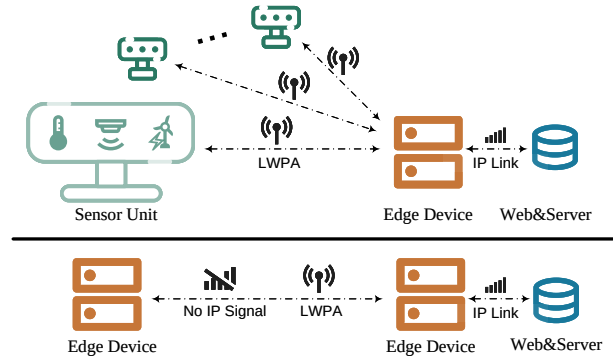


Figure 1: Time series data compression in two example situations.

Existing time series data compression methods primarily focus on storage and computational efficiency through temporal reduction, which can be categorized into three types. (1) Representation-based methods [5, 19, 28] use mathematical functions or learned nonlinear mappings to obtain compact latent representations. (2) Dictionary-based methods [22, 36] exploit structural redundancy via similarity measures or codebook matching. (3) Sequential algorithms [22, 24, 36, 37] process data incrementally using information-theoretic principles for online or model-free prediction. While compressing along the temporal dimension reduce input size, it can obscure crucial short-term patterns and make down-sampling a potentially risky operation for prediction task [51].



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792488>

On the other hand, general-purpose dimension-reduction techniques can be directly applied for channel compression, including Principal Component Analysis (PCA) [12], Multi-Dimensional Scaling (MDS) [40], Isomap [39], t-distributed Stochastic Neighbor Embedding (t-SNE) [43], and Uniform Manifold Approximation and Projection (UMAP) [25]. However, while these methods aim to either capture temporal inter-channel dependencies or perform intra-channel reduction, they are not specifically designed for time series prediction. As a result, they fail to integrate both inter- and intra-channel information in a unified manner, and do not preserve or reveal the underlying predictability of the data after compression. Additionally, many of these methods, originally developed for data visualization or non-regression tasks, are unable to accurately reconstruct the original multichannel structure from the predicted compressed representation.

The Multiple-Input Multiple-Output (MIMO) paradigm in signal processing and the recent success of MIMONet [26] in image compression bring new opportunities to build a compression-decompression framework for multichannel time series prediction. However, applying this idea in practice still poses significant challenges. **First**, preserving the potential predictive information is non-trivial in the compression process. The temporal dependence inherent in time series data stores predictive information and provides a foundation for accurate forecasting. However, such predictability cannot be guaranteed during compression when using current approaches, such as circular convolution with keys sampled from a normal distribution (as in HRR [34]) or a Bernoulli distribution (as in MIMONet). **Second**, oversimplified assumptions on data pose a critical challenge for accurate decompression. In particular, the assumption of normally distributed input—adopted in HRR—does not align well with the properties of real-world time series data. This mismatch can result in inadequate decompression modules and additional errors, such as intra-channel interference across timestamps and inter-channel interference within the same timestamp. **Third**, the overhead introduced by the compression-decompression processes may offset its efficiency benefits. While compressed data naturally leads to reduced computational and transmission time, the compression-decompression steps themselves may introduce additional overhead, potentially increasing overall time complexity.

To address these challenges, we first theoretically show that redundancy in multichannel time series can be exploited to design a compression method that preserves predictive information. Based on this insight, we propose a predictability-aware compression-decompression framework for multichannel time series prediction. The predictability is maintained by extracting the underlying seasonal patterns through Fourier transform. In this framework, the compression module uses a circular-key for single channel encoding and multichannel predictability alignment, while the decompression module applies the same circular-key matrix with orthogonality constraints and a residual block to mitigate reconstruction errors. Time-complexity analysis confirms its superior efficiency over direct multichannel predictors. Extensive experiments on six datasets and four backbone predictors demonstrate runtime reduction without sacrificing accuracy, even for data without strong periodicity (e.g., sensor drift data). A Cobb–Douglas-based performance index [6] jointly evaluates accuracy and efficiency, highlighting

our framework’s overall advantage and broad compatibility across diverse predictors.

Our main contributions are summarized as follows:

- **Observations and Theoretical Foundations:** We provide theoretical analyses that prove the predictability-awareness of our compression module, grounding the results in empirical observations and supporting analysis.
- **Compression–Decompression Framework:** We propose a compression–decompression framework for multichannel time-series prediction that minimizes reconstruction error during decompression. Time-complexity analysis shows that the framework achieves better time efficiency compared with a direct multichannel predictor.
- **Experimental Analysis:** We conduct extensive experiments demonstrating that the proposed framework achieves lower runtime and superior performance when jointly considering prediction accuracy and computational cost, while maintaining strong compatibility across diverse module architectures.

The following sections are organized into Related Work, Problem Definition, Observations and Theoretical Foundations, Compression and Decompression Framework, and Experiments. The Observations and Theoretical Foundations section shows how our method preserves predictability. The Compression and Decompression Framework section explains the details shown in Fig. 3 and describes overall process in detail.

2 Related Works

Time Series Prediction. Deep learning architectures for multichannel time series forecasting can be broadly categorized into several paradigms. Channel-independent models [32, 50] treat each channel separately, ignoring inter-series correlations to achieve scalability and simplicity. In contrast, channel-mixing models [2, 8, 14] explicitly model dependencies across different channel to improve predictive accuracy. Single-channel architectures [33] are originally designed for single channel inputs but can be generalized to multichannel settings. In addition, recent efforts have explored using large pre-trained models for time series forecasting. Time-LLM [16] reprograms frozen large language model (LLM) by encoding temporal data as token sequences to enable zero-shot forecasting. UniST [49] proposes a universal prompt-based spatio-temporal predictor that adapts to a wide range of urban forecasting tasks.

The growing use of deep learning and LLM in time series forecasting, which requires massive volumes of data, calls for compression methods that preserve predictive information under edge-device and sensor constraints.

Time Series Data Compression. There are two main aspects of time series compression: one along the temporal dimension and the other along the channel dimension. Existing time-series compression methods primarily focus on storage and computational efficiency through temporal reduction, which can be categorized into three types. (1) Representation-based methods [5, 19, 28] use mathematical functions or learned nonlinear mappings to obtain compact latent representations. (2) Dictionary-based methods [22, 36] exploit structural redundancy via similarity measures or codebook matching. (3) Sequential algorithms [22, 24, 36, 37] process

data incrementally using information-theoretic principles for on-line or model-free prediction. On the other hand, general-purpose dimension-reduction techniques—such as Principal Component Analysis (PCA) [12], Multi-Dimensional Scaling (MDS) [40], Isomap [39], t-distributed Stochastic Neighbor Embedding (t-SNE) [43], and Uniform Manifold Approximation and Projection (UMAP) [25]—are often applied for channel compression without task-specific design. For example, DeepGLO [38] adopt low-rank matrix factorization inspired by the principles of PCA to extract global features, although not specifically for time series data compression.

Multiple-Input Multiple-Output (MIMO). Techniques such as MIMO with Sparse Code Multiple Access [11, 18], Non-Orthogonal Multiple Access [7, 46], and Code Division Multiple Access [1, 21] have been proposed to allow different signals to share the same communication channel. A key principle in their design is to ensure that signals remain distinguishable and do not interfere with each other during transmission, typically through coding, modulation, or power allocation strategies. However, time series prediction requires preserving temporal context and inter-channel complementary information under compression transmission, which these methods do not support. Thus, while we are inspired by their channel sharing strategies, they are not directly applicable to predictive tasks.

3 Problem Definition

Before presenting the design of our method, we first describe the format and assumptions of the input multichannel time series data. We then introduce the objective function that guides our compression and prediction framework.

Definition 1: The number of input channels $C \in \mathbb{N}$ of the input time series data $X \in \mathbb{R}^{L \times C}$ refers to the input dimension that is distinct from the temporal axis with number of timestamps L .

Assumption 1: The multi-channel time series input $X_{t-L:t}^C \in \mathbb{R}^{L \times C}$ contains latent seasonal patterns, and the input length L is assumed to be longer than the most dominant seasonal period. Here, “seasonal period” denotes the dominant frequency component (e.g., via FFT), not a strict requirement for an obvious seasonal pattern in the data.

Objective Function. Motivated by the Cobb–Douglas production function [6], we define the objective function as a multiplicative function of prediction accuracy and runtime, as shown below:

$$\begin{aligned} \min_{\theta} \quad & \mathcal{F}_{\text{pred}}(\hat{X}_{t:t+H}^C, X_{t:t+H}^C) \cdot \mathcal{F}_{\text{time}}(\theta) \\ \text{s.t.} \quad & \dim(\text{Compressed}(X_{t-L:t}^C)) = (L, 1) \end{aligned} \quad (1)$$

In this formulation, θ denotes a set of parameters corresponding to the compression, prediction, and decompression modules. The variables $\hat{X}_{t:t+H}^C$ and $X_{t:t+H}^C$ represent the predicted and ground truth sequences for the next H time steps. The term $\mathcal{F}_{\text{pred}}$ quantifies the prediction error (e.g., Mean Squared Error), while $\mathcal{F}_{\text{time}}$ represents the total inference latency. The constraint enforces that the compressed representation maintains a single channel, i.e., $\dim = (L, 1)$.

In this design as shown in Fig. 3, a multichannel time series input $X_{t-L:t}^C \in \mathbb{R}^{L \times C}$ is first compressed into a single-channel

representation $X_{t-L:t} \in \mathbb{R}^{L \times 1}$. This compressed sequence is used to predict the next H time steps, resulting in $X_{t:t+H} \in \mathbb{R}^{H \times 1}$, which is then decompressed to reconstruct the multichannel output $X_{t:t+H}^C \in \mathbb{R}^{H \times C}$.

4 Observations and Theoretical Foundations

In this section, we first identify the redundancy phenomenon in multichannel time series data. Based on this observation, we present Theorem 1, which reveals the opportunity to remove redundant information while retaining predictive information during the compression process. Building on this theoretical opportunity, we introduce Theorem 2 to support that our proposed method is predictability-aware.

4.1 Redundancy in Multichannel Time Series

Multi-channel time series data can be represented by a limited number of channels. As shown in Table 1, the multichannel time series data across all datasets exhibit substantial redundancy. For example, in the *Sensor Drift* dataset, only 2 out of 321 principal components are sufficient to retain 95% of the total variance, and the top 50 components capture 100% of it. Similarly, *NYC taxi* and *DC bike* require fewer than 15 components to preserve 95% of the variance, with over 99% already explained by the top 50. Additionally, the first two components alone explain a dominant portion of the variance (e.g., 92.4% and 5.7% in *Sensor Drift*), indicating that the data lie near a low-dimensional manifold. These observations validate the presence of strong redundancy in the input channels and motivate the use of compact representations through channel compression.

Table 1: PCA summary across datasets. PCs @95% var.: Number of principal components needed to retain 95% of variance; Top 50 var.: Cumulative variance explained by top 50 PCs; PC1/PC2 var.: Variance explained by the 1st and 2nd principal components.

Metric	NYC_taxi	DC_bike	Sensor Drift
PCs @95% var.	15 / 321	13 / 321	2 / 321
Top 50 var.	0.9926	0.9908	1.0000
PC1 var.	0.5422	0.5723	0.9242
PC2 var.	0.1525	0.1993	0.0569

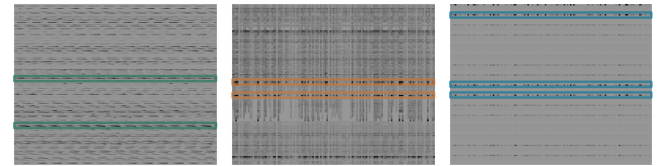


Figure 2: Heatmaps of raw multichannel time series data. The vertical axis represents 321 channels, and the horizontal axis denotes 321 time steps. Highlighted regions indicate examples of similar patterns.

Multi-channel time series data exists redundancy about similar pattern and low variation components. As shown in Figure 2, the heatmaps of the raw multichannel time series data reveal clear structural patterns across different datasets. A substantial number

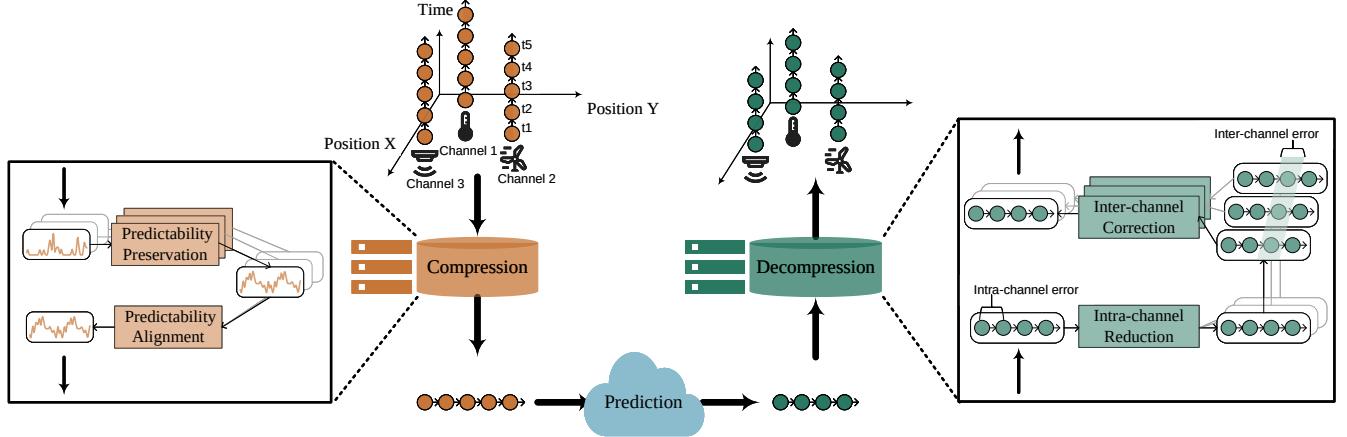


Figure 3: Overall architecture of the compression–decompression framework. Compression and decompression processes take place at the sensor or edge, while prediction is performed in the cloud. The framework is compatible with different deep learning prediction modules, such as transformer-based, MLP-based, and linear-based modules.

of channels exhibit little to no variation over time, as evidenced by large uniform-colored regions. In addition, multiple channels demonstrate similar temporal evolution patterns (such as the red box in Figure 2), suggesting redundancy and high correlation across channels. For example, in the DC bike dataset, certain areas show very few rentals. However, these data cannot be simply discarded, since there are still occasional rentals at specific times, although rentals are absent for most of the time. In addition, similar patterns can be observed in the DC dataset, which may arise from the same functional zones of the city or from nearby stations. These observations reveal the existence of compressible structure in data and justify an effective channel compression strategy prior to downstream prediction. To enable tractable modeling of such redundancy, the following assumption is made:

4.2 Retaining Predictive Information

Assumption 2: Gaussian redundancy assumption in multichannel time series. To model the redundancy or low-variation components often observed in multichannel time series data, we assume that these components follow a zero-mean multivariate Gaussian distribution. Specifically, the redundancy-induced uncertainty across all channels and time steps is represented as:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I_{CL}), \quad (2)$$

where $\varepsilon \in \mathbb{R}^{CL}$ is the vectorized noise across all channels and time steps, C is the number of channels, L is the sequence length, and I_{CL} is the identity matrix of size $CL \times CL$. Each per-channel noise vector $\varepsilon^{(i)} \in \mathbb{R}^L$ is assumed to be independent and identically distributed.

Motivated by Information Bottleneck (IB) [13], we have the following theorem on multi-channel time series.

Theorem 1: Upper bound on predictive information reduction under compression. Under Assumption 2, consider a compressed representation normalized to have unit signal variance. The IB admits the following upper bound:

$$\mathcal{L}_{IB} \leq \frac{1}{2} \log \left(1 + \frac{1}{CL \cdot \sigma^2} \right). \quad (3)$$

A detailed derivation is provided in the Appendix A.1.

Remark 1. This result highlights a fundamental trade-off: when the original number of channels C or the sequence length L is small, the upper bound on the IB loss becomes larger, indicating a higher risk of losing predictive information under compression. Conversely, larger C or L lead to a tighter bound, implying that high-dimensional inputs offer greater potential for safe and effective compression without sacrificing prediction accuracy. This insight theoretically supports the feasibility of compressing multichannel time series in scenarios with abundant channel or temporal resolution.

4.3 Predictability Awareness Compression

Circular convolution [34] provides a fixed-width vector operation for binding two representations and has been widely used as a method for compositional encoding in distributed representations. We treat this as a baseline mechanism for compression.

Definition 2: Correlation as a measure of predictability in temporal sequences. In time series data, the Pearson correlation coefficient (ranging from 0 to 1) between a historical segment and a future segment reflects the degree of predictability [17, 48], with higher correlation indicating stronger temporal dependency or clearly seasonal pattern. Specifically, for two aligned sequences $x, y \in \mathbb{R}^L$, a higher Pearson correlation coefficient implies that one sequence is more informative about the other.

Theorem 2: Circular keys preserve single channel predictability information. Given Assumption 1 and Definition 2, when the circular key $K \in \mathbb{R}^L$ shares the same seasonal period as the latent dominant seasonal period of the input sequence $X_{t-L:t}^C$, the circular convolution can retain the seasonal alignment and thereby preserve predictive information in the compressed representation Y .

$$Y^i = \text{Encode}(K^i, X^i) = X^i * K^i, \quad (4)$$

$$\text{s.t. } k_t = k_{t \bmod \tau}, \quad \text{for some fixed } \tau \ll L,$$

where the key enforces a repeating structure with seasonal period τ . The proof is provided in Appendix A.2.

Remark 2. A formal justification based on the Pearson correlation coefficient shows that when the seasonal key aligns with the latent seasonal period in the input, the encoded representation maximizes predictability, achieving a value of $R = 1$.

Remark 3. From a signal processing perspective, the seasonal circular key acts as a matched filter, amplifying frequency components that correspond to the latent seasonality in the input. This alignment ensures that the compressed representation Y emphasizes task-relevant information while filtering out redundant or irrelevant patterns.

Corollary 1: Inter-channel predictability alignment via Least Common Multiple Seasonal Period (LCMP). Given Theorem 2, consider a multichannel input $\{X^i\}_{i=1}^C$, where each channel $X^i \in \mathbb{R}^L$ exhibits a latent dominant seasonal pattern with seasonal period τ_i . To preserve inter-channel seasonal consistency and maintain the predictability of the combined representation, we define a shared seasonal period

$$\tau = \text{lcm}(\tau_1, \tau_2, \dots, \tau_C), \quad (5)$$

and constrain each circular key $K^i \in \mathbb{R}^L$ to satisfy $k_t^i = k_{t \bmod \tau}^i$. Each channel is then encoded as $Y^i = \text{Encode}(K^i, X^i)$, and the final compressed representation is the channel-wise summation:

$$Y = \sum_{i=0}^{C-1} Y^i = \sum_{i=0}^{C-1} \text{Encode}(K, X^i). \quad (6)$$

Remark 4. This seasonal shared-period encoding ensures that all encoded sequences remain aligned, enabling effective cross-channel compression and preserving the overall temporal structure in the latent representation. The seasonal keys K from different channels can be identical, since they follow the same seasonal period τ . In the next part, we will introduce a sparse method with reduced compression time, with minor limitations in predictability.

5 Compression and Decompression Framework

Based on the theoretical analysis above, this section illustrates the process of compression and decompression, as well as the time complexity of sparse optimization. As shown in Fig. 3, the framework is simple and flexibly compatible with existing time series prediction modules. The compression process has been justified in the former section with two steps (this section introduces a more unified and sparse method), and the following decompression process also involves two error-removal steps. Then, we prove that the framework with a single-channel predictor has lower time complexity than a direct multichannel predictor. Additional details on the pseudocode and training strategy are provided in the Appendix A.4.

5.1 Compression

Based on the former explanation and justification about predictability protection compression, the compression process is shown below. A sparse compression strategy is introduced to further reduce the computational burden on edge devices.

Concretely, each channel $X^i \in \mathbb{R}^L$ is partitioned into non-overlapping fragments of length τ , where τ is the least common period (LCP) across all channels. Only complete segments are encoded, and any remaining tail shorter than τ is omitted. The compressed

output is computed as:

$$Y = \sum_{i=0}^{C-1} \sum_{j=0}^{M-1} \text{Encode}(X_{j\tau:(j+1)\tau}^i, K), \quad (7)$$

where $M = \lfloor \frac{L}{\tau} \rfloor$, and K is the seasonal key shared across all segments. The time complexity of this sparse compression process is reduced from $O(CL^2)$ to $O\left(\frac{CL^2}{\tau}\right)$. When the period is chosen as $\tau = \frac{L}{\log L}$, the complexity becomes $O(CL \log L)$.

5.2 Decompression

The decompression process is designed to mitigate the reconstruction errors introduced by the simplifying assumptions in HRR, particularly the assumption that input data follows a Gaussian distribution—an assumption that often does not hold for real-world time series.

Decoding process via circular correlation. Inspired by HRR, the basic decoding process uses circular correlation with seasonal keys, defined as:

$$\begin{aligned} \tilde{X}^i &= \text{Decode}(K, \hat{Y}) \\ &= \left[\sum_{z=0}^{n-1} \hat{y}_z k_z, \sum_{z=0}^{n-1} \hat{y}_z k_{z-1}, \dots, \sum_{z=0}^{n-1} \hat{y}_z k_{z-h} \right], \end{aligned} \quad (8)$$

where $\hat{Y} = [\hat{y}_0, \hat{y}_1, \dots, \hat{y}_{h-1}]$ is the predicted representation from the forecasting model, h denotes the length of the predicted future sequence to be reconstructed, and k_z is the z -th element of the decoding kernel K . Under HRR, this yields:

$$\tilde{X}^i[t] \approx \left(\sum k^2 \right) \cdot \sum_{c=1}^C x_t^c + \eta_t, \quad (9)$$

where η_t denotes accumulated intra-channel interference due to overlapping shifts in the circular convolution.

Reducing intra-channel errors via orthogonal keys. To reduce intra-channel interference, we use orthogonal circular key matrices K , where each key shift is orthogonal to others: $K^\top K = I$. Such matrices can be efficiently constructed via two FFTs [45]. In this setting, different time shifts become mutually orthogonal, nullifying off-timestamp interference in decoding:

$$\tilde{X}^i[t] = \left(\sum k^2 \right) \cdot \sum_{c=1}^C x_t^c \quad (10)$$

Inter-channel error correction via residuals. While orthogonality mitigates intra-channel interference, inter-channel errors still remain. To address this, we introduce a residual correction block. The final decompression is expressed as:

$$\hat{X} = \text{Dense} \left(\frac{\text{Copy}(\tilde{X})}{\sum k^2} + \text{Residual} \left(\frac{\text{Copy}(\tilde{X})}{\sum k^2} \right) \right) \quad (11)$$

where $\tilde{X} = \text{Decode}(K, Y)$ with orthogonal circular key matrices, $\text{Copy}(\cdot)$ duplicates or projects the decoded result into the full channel dimension, $\text{Residual}(\cdot)$ denotes a lightweight correction block (2 conv layers + ReLU), $\text{Dense}(\cdot)$ is fully connected layer. This decomposition corrects for shared-channel entanglement and significantly enhances decompression fidelity.

5.3 Time Complexity Superiority

This part provides a time complexity analysis for compression and decompression framework.

Theorem 3: Computational Superiority Condition. Let $O(L)_{\text{comp}}$ denote the total computational complexity of our compression framework followed by a single-channel predictor, and let $O(L)_{\text{naive}}$ denote that of a naive multichannel predictor, which shares the same structure as the single-channel predictor except that it operates on all channels directly. Here, E denotes the number of training epochs and D denotes the depth of predictor. Then:

$$O(L)_{\text{comp}} < O(L)_{\text{naive}} \Rightarrow C > \frac{DE}{DE - 1 - \frac{1}{\tau}}. \quad (12)$$

5.3.1 Corollary 2: Simplified Superiority Condition. Under the asymptotic regime $DE \gg 1$ and $\frac{1}{\tau} \rightarrow 0$, the condition reduces to $C > 1$. The proof of Proposition 4 and Corollary is provided in Appendix A.3.

Remark 5. Theorem 3 and Corollary 2 together indicate that the proposed compression-decompression framework offers a computational advantage over the naive multichannel predictor when the number of input channels C exceeds one. Furthermore, the overhead introduced by compression and decompression becomes subdominant. As a result, the scaling advantage of the proposed framework remains intact in practical high-dimensional time series settings.

6 Experiments

In this section, we first introduce the experimental settings, including datasets, baselines, and evaluation metrics. We then analyze the results of the comparative experiments from different metrics aspect.

6.1 Experimental Settings

Datasets. We evaluate our design on datasets from a diverse set of edge-based sensing applications, including (1) traffic: NYC taxi [30] and DC_bike [3]; (2) energy: solar energy generation [29] and electricity consumption [41]; (3) manufacturing: gas sensor array drift [44]; and (4) weather [4].

Baselines and setup. We implement our framework with multiple existing predictors (MLP, Transformer, Linear, and Convolution, as detailed in the Appendix). Each such combination is referred to as an integration (e.g., PCDF-MLP, PCDF-Trans, PCDF-Linear, PCDF-Conv). The architectural details of each single-channel predictor are described as follows:

- **MLP integration:** Built upon the TSMixer-based MLP, this variant incorporates an additional flatten layer, one linear layer, and a ReLU activation, appended to the end of the original structure.
- **Transformer integration:** Based on the PatchTST encoder, this variant extends the original architecture by adding a flatten layer, two linear layers, and a ReLU activation.
- **Linear integration:** A linear predictor derived from HDMixer, consisting of multiple linear layers, with a flatten operation inserted before a final fully connected layer.

- **Convolution integration:** A convolutional module comprising two convolutional layers, followed by a flatten operation, two fully connected layers, and a ReLU activation function.

We compare these integrations with state-of-the-art models, including PatchTST [32], Autoformer [48], HDMixer [14], DLinear [50], MSGNet [2], and TSMixer [8], to evaluate performance and runtime trade-offs. Since, to the best of our knowledge, there is no existing multichannel compression framework specifically designed for time series prediction (as discussed in the Introduction and Related Work), no direct baselines are available for this setting. All models follow the same experimental setup, with the number of prediction channels set to $C \in \{5, 10, 20, 30, 40\}$. All experiments are conducted on two NVIDIA RTX 4090 GPUs, each equipped with 24 GB of memory.

Metric. Evaluation metrics include Mean Squared Error (MSE), runtime, and the Cobb–Douglas Production Index (CDPI). Runtime is obtained from the testing process, not from the training process; therefore, the number of training epochs does not affect the experimental results.

CDPI follows the idea of the Cobb–Douglas production function [6], which uses a multiplicative combination to model the joint contribution of multiple factors. We combine MSE and runtime in a similar fashion to capture the trade-off and enable a comprehensive analysis of prediction accuracy and computational efficiency.

$$\text{CDPI} = \text{MSE} \cdot \text{Runtime}. \quad (13)$$

In our experiments, we do not apply normalization when computing CDPI, in order to preserve the raw scale of MSE and runtime. Normalizing either metric before multiplication could introduce bias, as it implicitly assigns weights and may distort the true impact of each component.

Training. In this experiment, compression and decompression are trained on a single device. For real deployment, one option is centralized training with distributed inference, meaning that the model is trained in the cloud and then the model or parameters are transmitted to edge or sensor devices for compression and decompression. Another option is distributed training with distributed inference, where models are trained on different sensors or edge devices and both compression and decompression are performed locally on those devices.

6.2 CDPI Analysis

Based on the multiply of MSE and runtime results for all models (see details in Appendix A.5 and A.6), CPDI results are presented in Table 2. The compression–decompression integrations achieve lower combined index values compared to the representative baseline models across different datasets in most cases. These results suggest that the compression–decompression framework has the potential to achieve better overall performance by reducing prediction time and lowering prediction errors with the high compression ratio.

From the perspective of different datasets, those with clear seasonal patterns, such as Electricity, tend to exhibit lower combined index values. This indicates that such datasets either require less computational time or result in lower prediction errors after processing, leading to better overall performance. Conversely, datasets

Table 2: Cobb–Douglas Production Index (CDPI) results. Lower values indicate better overall performance. Underlined values denote the best performance in each setting. The number of channels corresponds to the compression ratio, as our framework compresses all channels into one.

Dataset & Channel		MLP Based Comparison			Transformer Based Comparison			Linear Based Comparison		
		PCDF-MLP	TSMixer	Ratio (MLP/TS)	PCDF-Trans	PatchTST	Ratio (Trans/Patch)	PCDF-Linear	HDMixer	Ratio (Linear/HD)
NYC taxi 0.1/24	5	0.000459	0.000397	1.156	0.000340	0.000716	0.475	0.000310	0.002464	0.126
	10	<u>0.000524</u>	0.000567	0.924	<u>0.000386</u>	0.001026	0.376	<u>0.000348</u>	0.003966	0.088
	20	0.000581	0.000735	0.790	0.000478	0.001853	0.258	0.000968	0.006231	0.155
	30	<u>0.000664</u>	0.000885	0.750	<u>0.000564</u>	0.002532	0.223	<u>0.000528</u>	0.007694	0.069
	40	<u>0.000761</u>	0.000969	0.785	<u>0.000745</u>	0.003158	0.236	<u>0.000632</u>	0.012362	0.051
DC_bike 0.3/30	5	0.000539	0.000647	0.833	0.000413	0.001063	0.389	0.000505	0.009265	0.055
	10	<u>0.000619</u>	0.000864	0.716	<u>0.000414</u>	0.001693	0.244	<u>0.000366</u>	0.016486	0.022
	20	0.000691	0.001150	0.601	<u>0.000643</u>	0.003208	0.200	<u>0.000543</u>	0.028265	0.019
	30	<u>0.000804</u>	0.001345	0.598	<u>0.000949</u>	0.003812	0.249	<u>0.000622</u>	0.037110	0.017
	40	<u>0.000984</u>	0.001500	0.656	<u>0.001235</u>	0.005163	0.239	<u>0.003392</u>	0.050074	0.068
Electricity 0.1/24	5	0.000299	0.000215	1.391	0.000219	0.000337	0.650	0.000169	0.001188	0.142
	10	<u>0.000242</u>	0.000285	0.849	<u>0.000234</u>	0.000611	0.383	<u>0.000195</u>	0.001984	0.098
	20	<u>0.000394</u>	0.000448	0.879	<u>0.000319</u>	0.000972	0.328	<u>0.000335</u>	0.004585	0.073
	30	<u>0.000430</u>	0.000523	0.822	<u>0.000404</u>	0.001315	0.307	<u>0.000312</u>	0.007746	0.040
	40	<u>0.000525</u>	0.000697	0.753	<u>0.000534</u>	0.001740	0.307	<u>0.000425</u>	0.013574	0.031
Solar Energy 0.25/24	5	0.000765	0.000980	0.781	0.000538	0.001336	0.403	0.000508	0.007004	0.073
	10	<u>0.000586</u>	0.000915	0.641	<u>0.000506</u>	0.001576	0.321	<u>0.000416</u>	0.007520	0.055
	20	<u>0.000631</u>	0.001087	0.581	<u>0.000541</u>	0.002316	0.234	<u>0.000512</u>	0.015104	0.034
	30	<u>0.000730</u>	0.001384	0.527	<u>0.000625</u>	0.003065	0.204	<u>0.000596</u>	0.026226	0.023
	40	<u>0.000857</u>	0.001401	0.612	<u>0.000750</u>	0.003609	0.208	<u>0.000869</u>	0.039396	0.022
Sensor Drift 0.1/30	5	0.000548	0.001070	0.512	0.000434	0.001573	0.276	0.000368	0.010083	0.037
	10	<u>0.000638</u>	0.001385	0.461	<u>0.000456</u>	0.002456	0.186	<u>0.000472</u>	0.013823	0.034
	20	<u>0.000846</u>	0.002084	0.406	<u>0.000666</u>	0.004751	0.140	<u>0.000792</u>	0.024024	0.033
	30	<u>0.000951</u>	0.002270	0.419	<u>0.000841</u>	0.006407	0.131	<u>0.000861</u>	0.031081	0.028
	40	<u>0.001077</u>	0.002886	0.373	<u>0.000929</u>	0.009078	0.102	<u>0.000850</u>	0.033141	0.026
Weather 0.1/25	5	0.000342	0.000434	0.788	0.000273	0.000989	0.276	0.000425	0.001812	0.235
	10	<u>0.000394</u>	0.000547	0.720	<u>0.000306</u>	0.001193	0.257	<u>0.000267</u>	0.003822	0.070
	20	<u>0.000569</u>	0.000731	0.778	<u>0.000382</u>	0.002106	0.182	<u>0.000391</u>	0.007379	0.053
	30	<u>0.000590</u>	0.000883	0.668	<u>0.000535</u>	0.003086	0.173	<u>0.000475</u>	0.011419	0.042
	40	<u>0.000630</u>	0.001081	0.583	<u>0.000528</u>	0.004326	0.122	<u>0.000533</u>	0.022849	0.023

with less pronounced seasonal patterns tend to have higher combined index values, reflecting increased prediction errors or longer computational times.

Table 3: Training time (in seconds) under different train horizons

Method	channel=5	channel=10	channel=20	channel=40
PCDF-MLP	0.17080	0.14992	0.17723	0.58769
PCDF-Trans	0.25910	0.24673	0.27429	0.67120
PCDF-Linear	0.16091	0.17340	0.20060	0.48645
PatchTST	0.63854	0.92027	1.47746	3.64563
AutoFormer	1.53149	1.52916	1.48094	2.48258
HDMixer	0.88682	1.29958	2.06925	4.35996
Dlinear	0.07111	0.12328	0.22287	0.80155

6.3 Runtime and Accuracy Analysis

Figure 4 shows the variance of MSE. The integrations exhibit lower variance across different datasets compared to their corresponding base models, indicating that our framework provides better stability for the underlying predictors.

In terms of overall MSE and runtime values, the integrations generally achieve lower values than their base models, except for

the PCDF-MLP and its corresponding base model TSMixer in the MSE metric. This suggests that our framework can effectively reduce runtime, decrease MSE, and improve overall performance compared to other methods built upon the same predictors, despite the exception observed with TSMixer.

Table 4: Compression-decompression framework ablation study via MSE (values rounded to 4 decimal places). Abbreviations: Comp. = Compression, Decomp. = Decompression, Decode. = Decoder, Encode. = Encoder.

Method	NYC taxi	DC bike	Electricity	Solar Energy	Sensor Drift
Comp.-Decomp.	0.1800	0.2153	0.1073	0.2172	0.2389
Rand.-Decomp.	0.1983	0.2595	14.8035	18.5596	0.2533
Comp.-Decode.	0.2132	1257.9678	0.2435	9.8324	0.5709
Encode.-Decomp.	0.3232	5.4924	143.5187	1.9980	12.7464
Encode.-Decode.	0.1546	1.5364	0.1737	1.6343	0.3028

6.4 Training Time Analysis

Table 3 shows the training time of the proposed framework under the same number of epochs. The PCDF-family scales well with channel count, with slower growth in training time—PCDF-Trans

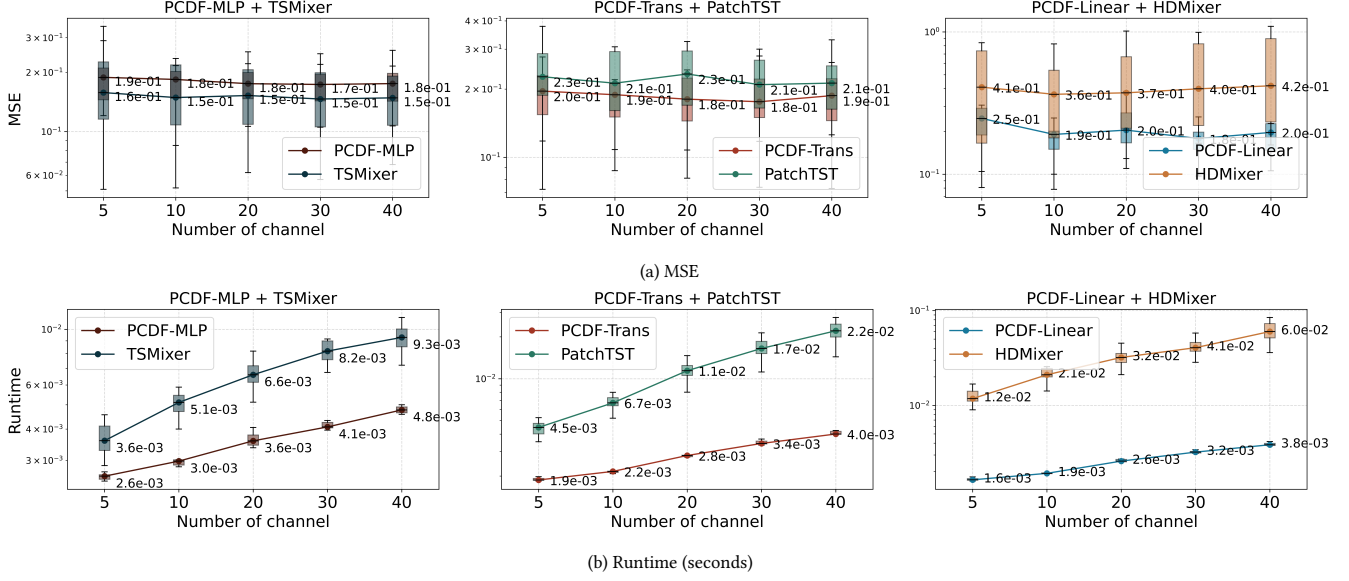


Figure 4: Boxplot of MSE and runtime across channel configurations and datasets

($\times 2.6$), PCDF-Linear ($\times 3.0$), and PCDF-MLP ($\times 3.4$)—compared to PatchTST ($\times 5.7$), HDMixer ($\times 4.9$), and DLinear ($\times 11.3$). Even with 40 channels, all PCDF variants finish training in less than one second, showing clear speed-ups over methods without the compression-decompression framework. These results indicate that the proposed methods become more efficient when the number of input channels increases.

6.5 Ablation Analysis

In the ablation study, a classic encoder-decoder framework is used to replace the compression and decompression components in order to justify the effectiveness of the proposed framework. In addition, a random key is used in place of the seasonal key to evaluate the effectiveness of the compression component guided by seasonal information. Therefore, the ablation study includes four variants: (1) random-key compression-decompression, (2) compression-decoder, (3) encoder-decompression, and (4) encoder-decoder.

In the random-key variant, the results vary significantly across datasets, suggesting that the random key fails to uncover the underlying predictive patterns in some datasets. This highlights the importance of using the seasonal key within the compression-decompression framework.

For the compression decoder and encoder decompression variants, since compression and decompression are co-designed and interdependent, using only one of the components leads to unstable results. This indicates that optimal performance relies on the synergy between both compression and decompression modules.

In the Encoder-Decoder variant, although lower MSE is observed on the NYC Taxi dataset, the performance remains unstable overall. This may be attributed to the fact that the encoder cannot effectively preserve or extract predictive patterns, and the corresponding decoder may fail to recover the single-channel signal in a lossless manner. The inferior and unstable results of these ablation variants validate the superiority of our framework in both extracting

latent predictive information during compression and performing accurate reconstruction during decompression.

7 Conclusion

This paper proposes a novel predictability-aware compression-decompression framework tailored for multichannel time series prediction. By leveraging a seasonality-guided convolution process with circularly orthogonal key matrices, the framework effectively preserves latent temporal dependencies during compression and mitigates reconstruction errors during decompression. Extensive experiments across diverse datasets and prediction backbones demonstrate that our framework consistently achieves lower runtime and improved prediction accuracy, as evidenced by reduced Cobb-Douglas production indices. Overall, this work offers a flexible and efficient solution for compressing multichannel time series data without sacrificing predictive performance, paving the way for more resource-efficient forecasting in edge-cloud environments.

Limitations and future work. Despite strong model compatibility, the framework requires retraining when applied to new domains, limiting its out-of-domain generalization. Additionally, under extreme compression ratios, training instability such as gradient explosion may occur. While this is alleviated via scale-invariant gradient clipping, we attribute the root cause to the large dimensional disparity between compressed and decompressed representations. Future work will explore principled mechanisms to bridge this gap and enhance training robustness.

Acknowledgments

We acknowledge the support of NSF (2520547). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

- [1] Bahidja Boukenadil, Bendjillali Ridha Ilyas, Mohammed Sofiane Bendelhoum, Tadjeddine Ali Abderrazak, and Kamline Miloud. 2024. Deep learning-driven channel coding: enhancing performance in LDS-CDMA and NOMA systems. *STUDIES IN ENGINEERING AND EXACT SCIENCES* (2024). <https://doi.org/10.54021/see5v5n2-325>
- [2] W. Cai et al. 2024. MSGNet: Learning Multi-Scale Inter-series Correlations for Multivariate Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 11141–11149.
- [3] Capital Bikeshare. 2025. Capital Bikeshare Trip History Data. <https://s3.amazonaws.com/capitalbikeshare-data/index.html>. Accessed: 2025-05-03.
- [4] J. Cejpek and Meteostat contributors. 2025. Meteostat: Historical Weather and Climate Data. <https://meteostat.net/>. Accessed: 2025-05-15.
- [5] Shubham Chandak, Amirhossein Mirhoseini, and Tsachy Weissman. 2020. LFZip: Lossy Compression of Multivariate Floating-Point Time Series Data via Improved Prediction. In *Data Compression Conference (DCC)*. IEEE, 342–351.
- [6] C. W. Cobb et al. 1928. A Theory of Production. *The American Economic Review* 18, 1 (1928), 139–165.
- [7] Jiefei Ding and Jun Cai. 2017. Efficient MIMO-NOMA Clustering Integrating Joint Beamforming and Power Allocation. In *IEEE Global Communications Conference (GLOBECOM)*. <https://doi.org/10.1109/GLOCOM.2017.8254639>
- [8] V. Ekambaram et al. 2023. TSMixer: Lightweight MLP-Mixer Model for Multivariate Time Series Forecasting. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 459–469.
- [9] W. Fan et al. 2023. Dish-TS: A General Paradigm for Alleviating Distribution Shift in Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 7522–7529.
- [10] Atonu Ghosh, Mayank Raj, Laksh Bhatia, Sandeep Singh, Munish Kumar, and Ramesh K. Jha. 2023. LoRaute: Routing Messages in Backhaul LoRa Networks for Underserved Regions. *IEEE Internet of Things Journal* 10, 22 (2023), 19964–19971. <https://doi.org/10.1109/IJOT.2023.3287102>
- [11] Shuai Han, Cheng Guo, W. Meng, Cheng Li, Yang Cui, and Wenyan Tang. 2016. The uplink and downlink design of MIMO-SCMA system. In *International Wireless Communications and Mobile Computing Conference (IWCMC)*. <https://doi.org/10.1109/IWCMC.2016.7577033>
- [12] H. Hotelling et al. 1933. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology* 24, 6 (1933), 417–441.
- [13] Shizhe Hu, Zhengzheng Lou, Xiaoqiang Yan, and Yangdong Ye. 2024. A Survey on Information Bottleneck. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 8 (2024), 5325–5344. <https://doi.org/10.1109/TPAMI.2024.3366349>
- [14] Q. Huang et al. 2024. HDMixer: Hierarchical Dependency with Extendable Patch for Multivariate Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 12608–12616.
- [15] K. Jang et al. 2025. Explainable Artificial Intelligence for Fault Diagnosis of Industrial Processes. *IEEE Transactions on Industrial Informatics* 21, 1 (2025), 4–11.
- [16] Ming Jin, Zheng Sun, Weize Chen, Yiming Li, and Yu Zheng. 2024. TimeLLM: Time Series Forecasting by Reprogramming Large Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VhXyR2Mf53>
- [17] A. Khelifati et al. 2019. Corad: Correlation-aware Compression of Massive Time Series Using Sparse Dictionary Coding. *IEEE Big Data* (2019), 2289–2298.
- [18] Alva Kosasih, O. Setyawati, and R. Rahmadwati. 2018. Low Complexity Multi-User MIMO Detection for Uplink SCMA System Using Expectation Propagation Algorithm. *TELKOMNIKA* 16 (2018), 182–188. <https://doi.org/10.12928/TELKOMNIKA.V16I1.7365>
- [19] Li Li, Xiaolei Ma, Zhaohui Wang, and Feng Chen. 2014. Traffic prediction, data compression, abnormal data detection and missing data imputation: An integrated study based on the decomposition of traffic time series. In *17th International IEEE Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 282–289.
- [20] Z. Li et al. 2022. Robust Training of Neural Networks Using Scale Invariant Architectures. In *Proceedings of the International Conference on Machine Learning (ICML)*. 12656–12684.
- [21] Hancheng Liao. 2007. MIMO-LAS-CDMA System with Multiple Antennas. In *IEEE International Conference on Wireless Communications, Networking and Mobile Computing (WiCOM)*. <https://doi.org/10.1109/WICOM.2007.31>
- [22] J. M. Lillo-Castellano, F. Luna, A. M. Garcia, and M. Rosa-Zurera. 2013. Weaning outcome prediction from heterogeneous time series using Normalized Compression Distance and Multidimensional Scaling. *Expert Systems with Applications* 40, 5 (2013), 1737–1747.
- [23] X. Liu et al. 2023. LargeST: A Benchmark Dataset for Large-Scale Traffic Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [24] A. S. Lysyak and B. Ya. Ryabko. 2016. Time series prediction based on data compression methods. *Problems of Information Transmission* 52, 1 (2016), 92–99.
- [25] L. McInnes et al. 2018. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. [arXiv:1802.03426](https://arxiv.org/abs/1802.03426) [stat.ML]
- [26] N. Menet et al. 2023. MIMONets: Multiple-Input-Multiple-Output Neural Networks Exploiting Computation in Superposition. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [27] Christos Milarokostas, Nadeem Javaid, Thrasyvoulos Spyropoulos, and Symeon Papavassiliou. 2023. A Comprehensive Study on LPWANs With a Focus on the Potential of LoRa/LoRaWAN Systems. *IEEE Communications Surveys & Tutorials* 25, 1 (2023), 825–867. <https://doi.org/10.1109/COMST.2022.3222922>
- [28] Aekyung Moon, Taesu Kim, Jaehong Kim, and Byoung-Tak Zhang. 2022. Compression and regularized optimization of modules stacked residual deep fuzzy system with application to time series prediction. *Information Sciences* 608 (2022), 551–577.
- [29] National Renewable Energy Laboratory. 2025. Solar Power Data for Integration Studies. <https://www.nrel.gov/grid/solar-power-data.html>. Accessed: 2025-05-04.
- [30] New York City Taxi and Limousine Commission. 2025. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Accessed: 2025-05-01.
- [31] T. Nguyen et al. 2024. Scaling Transformer Neural Networks for Skillful and Reliable Medium-Range Weather Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [32] Y. Nie et al. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations (ICLR)*.
- [33] S. Palaskar et al. 2024. AutoMixer for Improved Multivariate Time-Series Forecasting on Business and IT Observability Data. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 22962–22968.
- [34] T. A. Plate. 1995. Holographic Reduced Representations. *IEEE Transactions on Neural Networks* 6, 3 (1995), 623–641.
- [35] Usman Raza, Parag Kulkarni, and Mahesh Sooriyabandara. 2017. Low Power Wide Area Networks: An Overview. *IEEE Communications Surveys & Tutorials* 19, 2 (2017), 855–873. <https://doi.org/10.1109/COMST.2017.2652320>
- [36] Boris Ryabko. 2009. Compression-based methods for nonparametric prediction and estimation of some characteristics of time series. *IEEE Transactions on Information Theory* 55, 9 (2009), 4309–4315.
- [37] Boris Ryabko, Jaakko Astola, and Alexey Gammerman. 2016. *Compression-Based Methods of Statistical Analysis and Prediction of Time Series*. Springer.
- [38] R. Sen et al. 2019. Think Globally, Act Locally: A Deep Neural Network Approach to High-Dimensional Time Series Forecasting. *Advances in Neural Information Processing Systems* 32 (2019).
- [39] J. B. Tenenbaum et al. 2000. A Global Geometric Framework for Nonlinear Dimensionality Reduction. *Science* 290 (2000), 2319–2323.
- [40] W. S. Torgerson et al. 1952. Multidimensional Scaling: I. Theory and Method. *Psychometrika* 17, 4 (1952), 401–419.
- [41] Artur Trindade. 2015. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C58C86>.
- [42] S. Tu et al. 2024. PowerPM: Foundation Model for Power Systems. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [43] L. van der Maaten and G. Hinton. 2008. Visualizing Data using t-SNE. *Journal of Machine Learning Research* (2008), 2579–2605.
- [44] A. Vergara et al. 2012. Chemical Gas Sensor Drift Compensation Using Classifier Ensembles. *Sensors and Actuators B: Chemical* 166–167 (2012), 320–329.
- [45] R. Wang et al. 2018. Orthogonal Circulant Structure and Chaotic Phase Modulation based Analog to Information Conversion. *Signal Processing* 144 (2018), 104–117.
- [46] Taotao Wang, Long Shi, K. Cai, Lining Tian, and Shengli Zhang. 2020. Non-Coherent NOMA With Massive MIMO. *IEEE Wireless Communications Letters* 9 (2020), 134–138. <https://doi.org/10.1109/LWC.2019.2944620>
- [47] Dixin Wu, Ziyang Zhou, Bing Zhou, Hongwei Li, and Kim-Kwang Raymond Choo. 2023. A Low-Cost Low-Power LoRa Mesh Network for Large-Scale Environmental Sensing. *IEEE Internet of Things Journal* 10, 19 (2023), 16700–16714. <https://doi.org/10.1109/IJOT.2023.3256279>
- [48] H. Wu et al. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*. 22419–22430.
- [49] Yuan Yuan, Juncheng Li, Lihao Li, and Jiebo Luo. 2024. UniST: A Prompt-Empowered Universal Model for Urban Spatio-Temporal Prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. ACM. <https://arxiv.org/abs/2406.03805>
- [50] A. Zeng et al. 2023. Are Transformers Effective for Time Series Forecasting?. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 11121–11128.
- [51] Siying Zhu, Jiawei Zheng, and Qianli Ma. 2025. MR-Transformer: Multiresolution Transformer for Multivariate Time Series Prediction. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2025), 1171–1183. <https://doi.org/10.1109/TNNLS.2023.3327416>

A Appendix

A.1 Upper Bound Derivation of the Information Bottleneck Objective

We adopt the Information Bottleneck (IB) principle [13], with the objective formulated as:

$$\mathcal{L}_{\text{IB}} = I(Y; X_{t:t-L}^C) - \beta I(Y; X_{t:t+H}^C), \quad (14)$$

where $X_{t:t-L}^C \in \mathbb{R}^{C \times L}$ is the historical input, $X_{t:t+H}^C \in \mathbb{R}^{C \times H}$ is the future target to predict, $Y \in \mathbb{R}^L$ is the compressed representation derived from $X_{t:t-L}^C$, and $\beta > 0$ is a trade-off parameter balancing compression and prediction.

Upper Bound on $I(Y; X_{t:t-L}^C)$. Following Assumption 2, we assume the historical input includes Gaussian redundancy:

$$X_{t:t-L}^C = S + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_{CL}), \quad (15)$$

where $S \in \mathbb{R}^{C \times L}$ is the true signal and $\varepsilon \in \mathbb{R}^{CL}$ is i.i.d. noise. We apply a linear compression:

$$Y = KX_{t:t-L}^C = KS + K\varepsilon, \quad (16)$$

where $K \in \mathbb{R}^{L \times CL}$ is the projection matrix. Assuming a Gaussian encoder, the mutual information upper bound becomes:

$$I(Y; X_{t:t-L}^C) \leq \frac{1}{2} \log \left(1 + \frac{1}{CL \cdot \sigma^2} \right). \quad (17)$$

Lower Bound on $I(Y; X_{t:t+H}^C)$. We now bound the prediction term using a variational approximation $q(x | y)$ of the true conditional:

$$\begin{aligned} I(Y; X_{t:t+H}^C) &= \mathbb{E}_{p(y, \hat{x})} [\log q(\hat{x} | y)] + \\ &\quad \underbrace{\mathbb{E}_{p(y, \hat{x})} \left[\log \frac{p(\hat{x} | y)}{q(\hat{x} | y)} \right]}_{\text{KL divergence} \geq 0} + H(X_{t:t+H}^C) \end{aligned} \quad (18)$$

Hence, the variational lower bound is:

$$I(Y; X_{t:t+H}^C) \geq \mathbb{E}_{p(y, \hat{x})} [\log q(\hat{x} | y)] + H(X_{t:t+H}^C). \quad (19)$$

If the model is well trained, $q(\hat{x} | y) \approx p(\hat{x} | y)$, and:

$$I(Y; X_{t:t+H}^C) \approx H(\hat{X}_{t:t+H}^C) - H(X_{t:t+H}^C) = 0. \quad (20)$$

Final Upper Bound. Combining both bounds, we obtain:

$$\mathcal{L}_{\text{IB}} \leq \frac{1}{2} \log \left(1 + \frac{1}{CL \cdot \sigma^2} \right) \quad (21)$$

This shows that increasing channel count C , history length L , or noise variance σ^2 will decrease the IB objective, promoting stronger compression.

A.2 Predictability Preservation in Compression

We provide a formal justification for how circular convolution preserves temporal predictability when the key structure aligns with the latent seasonality of the input.

Let each channel $X^i \in \mathbb{R}^L$ be partitioned into m non-overlapping segments of length τ , such that $L = m\tau$. Let the key $K \in \mathbb{R}^\tau$ be a seasonal vector with period τ , extended to length L via circular repetition.

Each compressed channel is encoded as:

$$Y^i[t] = \text{Encode}(K, X^i)[t] = \sum_{z=0}^{L-1} x_z^i \cdot k_{(t-z) \bmod L}, \quad (22)$$

$$t \in \{0, 1, \dots, L-1\}.$$

We can express the first τ outputs compactly as:

$$Y_{0:\tau}^i = \begin{bmatrix} K^{(0)} \odot X^i \\ K^{(1)} \odot X^i \\ \vdots \\ K^{(\tau-1)} \odot X^i \end{bmatrix}, \quad (23)$$

where $K^{(t)}$ denotes the circular shift of K by t positions, and $\odot$ denotes element-wise multiplication followed by summation.

To assess the preservation of seasonal structure, we compute the Pearson correlation coefficient between $Y^i[t]$ and its delayed version $Y^i[t + \tau]$:

$$R = \frac{\sum_{t=0}^{\tau-1} (Y^i[t] - \bar{Y}_{0:\tau}^i) (Y^i[t + \tau] - \bar{Y}_{\tau:2\tau}^i)}{\sqrt{\sum_{t=0}^{\tau-1} (Y^i[t] - \bar{Y}_{0:\tau}^i)^2} \cdot \sqrt{\sum_{t=0}^{\tau-1} (Y^i[t + \tau] - \bar{Y}_{\tau:2\tau}^i)^2}}, \quad (24)$$

where $\bar{Y}_{0:\tau}^i$ and $\bar{Y}_{\tau:2\tau}^i$ denote the mean values of the first and second seasonal segments of the encoded signal Y^i , respectively. This formulation evaluates the seasonal correlation between two adjacent cycles in the compressed output. Then $K^{(t)} = K^{(t+\tau)}$:

$$R = \frac{\sum_{t=0}^{\tau-1} (Y^i[t] - \bar{Y}^i)^2}{\sqrt{\sum_{t=0}^{\tau-1} (Y^i[t] - \bar{Y}^i)^2} \cdot \sqrt{\sum_{t=0}^{\tau-1} (Y^i[t] - \bar{Y}^i)^2}} = 1. \quad (25)$$

Hence, when the circular key aligns with the latent seasonal period τ , the circular convolution operation effectively preserves temporal dependencies in the encoded output, ensuring the retention of predictive information.

A.3 Time Complexity Justification

The time complexity of the proposed framework is derived by considering the complete pipeline, including **compression**, **decompression**, and **prediction**. The total computational complexity is given by:

$$\begin{aligned} O(L)_{\text{comp}} &= \underbrace{DEL^2}_{\text{Prediction}} + \underbrace{\frac{CL^2}{\tau} + 2L \log L}_{\text{Compression}} \\ &\quad + \underbrace{L + 3CL + CL^2 + CL^2}_{\text{Decompression}} \\ &\approx DEL^2 + \frac{CL^2}{\tau} + CL^2, \end{aligned} \quad (26)$$

where E denotes the number of training epochs, L is the number of timestamps, C ($C < L$) is the number of channels, D is the depth of the predictor, and τ is the least common multiple (LCM) seasonal period across different channels. In the asymptotic approximation, we neglect lower-order terms.

The time complexity of the decompression module, dominated by residual blocks, can be broken down into standard neural network operations:

Table 5: MSE comparison across models

Dataset & Channel		PCDF-MLP	PCDF-Conv	PCDF-Trans	PCDF-Linear	PatchTST	Autoformer	HDMixer	Dlinear	MSGNET	TSMixer
NYC taxi 0.1/24	5	0.17001	0.17123	0.17185	0.17617	0.18207	0.22242	0.21746	0.22952	0.17786	0.11583
	10	0.17345	0.17402	0.17866	0.18100	0.16052	0.21227	0.19901	0.22148	0.18730	0.10712
	20	0.16690	0.16755	0.17004	0.38246	0.17685	0.20932	0.20868	0.23443	0.15630	0.10621
	30	0.16273	0.18081	0.16454	0.16447	0.16722	0.19818	0.20644	0.23384	0.16985	0.10325
	40	0.16375	0.17950	0.18567	0.16808	0.15947	0.20778	0.21135	0.23004	0.16840	0.10497
DC bike 0.3/30	5	0.20727	0.21891	0.21993	0.30601	0.24833	0.25229	0.84002	0.33392	0.28390	0.19984
	10	0.20484	0.32077	0.19908	0.19993	0.26378	0.25559	0.82432	0.35534	0.24590	0.18705
	20	0.20499	0.20131	0.23476	0.20643	0.29622	0.26901	1.01525	0.34106	0.24141	0.18827
	30	0.19953	0.19884	0.27986	0.19940	0.25127	0.24671	0.99303	0.32456	0.24285	0.17986
	40	0.20115	0.20429	0.26227	0.81534	0.25409	0.29385	1.01694	0.33351	0.24126	0.18116
Electricity 0.1/24	5	0.12057	0.11257	0.11797	0.10462	0.07222	0.13478	0.08058	0.06247	0.06887	0.05089
	10	0.08506	0.10490	0.10813	0.10000	0.08733	0.13689	0.07827	0.05899	0.06863	0.05172
	20	0.10636	0.10162	0.10774	0.10971	0.08091	0.14275	0.12867	0.06489	0.07619	0.06189
	30	0.10527	0.09370	0.11765	0.09699	0.07376	0.14071	0.16501	0.06934	0.07419	0.05711
	40	0.10686	0.11419	0.12568	0.10561	0.07290	0.14487	0.16060	0.07126	0.07742	0.06797
Solar energy 0.25/24	5	0.28963	0.34007	0.27720	0.29853	0.37834	0.32659	0.78084	0.41998	0.30821	0.34265
	10	0.19477	0.19823	0.22016	0.20110	0.30252	0.26677	0.52998	0.32197	0.23241	0.22869
	20	0.18351	0.20605	0.19106	0.20172	0.28950	0.28373	0.71582	0.30441	0.22477	0.21239
	30	0.18445	0.19313	0.18724	0.19172	0.27392	0.27739	0.92214	0.28723	0.21932	0.20562
	40	0.18703	0.18488	0.18851	0.22520	0.25131	0.28551	1.09463	0.27789	0.21166	0.19491
Sensor drift 0.1/30	5	0.21170	0.24100	0.23216	0.22586	0.29965	0.29230	0.60090	0.49438	0.26063	0.23470
	10	0.21635	0.22153	0.21592	0.24843	0.30743	0.28553	0.54017	0.49369	0.25783	0.23546
	20	0.22143	0.26097	0.24296	0.29123	0.32431	0.30663	0.53079	0.52499	0.26535	0.25440
	30	0.21964	0.31027	0.23310	0.25249	0.30008	0.29319	0.53616	0.57539	0.26360	0.24865
	40	0.21537	0.25694	0.23342	0.22679	0.32976	0.29579	0.53791	0.57626	0.26061	0.25885
Weather 0.1/25	5	0.13686	0.13677	0.14842	0.26743	0.20510	0.20085	0.14801	0.15422	0.19025	0.11550
	10	0.13915	0.19315	0.14120	0.13915	0.16099	0.18119	0.17340	0.15355	0.16385	0.11092
	20	0.14017	0.14220	0.13612	0.15410	0.16739	0.18602	0.21575	0.17785	0.16017	0.11679
	30	0.13686	0.13609	0.14488	0.14433	0.16433	0.19715	0.26048	0.16975	0.16483	0.11303
	40	0.13583	0.13694	0.13162	0.13908	0.17373	0.19966	0.29841	0.16809	0.16277	0.11571

$$O(L)_{\text{decompression}} = L + 3CL + CL^2 + CL^2 \quad (27)$$

In contrast, a naive multichannel predictor that processes all channels jointly without compression incurs the following computational complexity:

$$O(L)_{\text{naive}} = DECL^2 \quad (28)$$

To determine the minimum number of channels C for which the proposed method achieves lower computational cost than the naive baseline, we compare both complexities:

$$\begin{aligned} O(L)_{\text{naive}} &> O(L)_{\text{comp}} \\ DECL^2 &> DEL^2 + CL^2 + \frac{CL^2}{\tau} \\ \Rightarrow C &> \frac{DE}{DE - 1 - \frac{1}{\tau}} \end{aligned} \quad (29)$$

Under the assumptions that $DE \gg 1$ and $\frac{1}{\tau} \rightarrow 0$, the lower bound simplifies to:

$$C > 1 \quad (30)$$

This result indicates that the proposed compression-based framework offers a computational advantage when the number of channels exceeds one. Furthermore, since the additional costs of compression and decompression are subdominant in the asymptotic regime, they do not alter the scaling benefit of the proposed method.

A.4 Framework Pipeline with Pseudocode

Pseudocode for training process. The pseudocode in Algorithm 1 describes the training phase of our compression–prediction–decompression framework.

Additional techniques for training the model. Following Dish-TS [9], we apply normalization after compression and denormalization before decompression.

Training loss functions. The total training objective consists of three terms:

$$\begin{aligned} \mathcal{L} = & \underbrace{\|\hat{X} - X\|^2}_{\text{Prediction Loss}} + \alpha \underbrace{\left(\sum_{i=1}^C |\text{Residual}_i| - \sum_{i=1}^C \hat{X}_i \right)^2}_{\text{Regulation Term}} \\ & + \beta \underbrace{\|\varphi_Y - \varphi_{\hat{Y}}\|^2}_{\text{Latent Consistency}}. \end{aligned} \quad (31)$$

Here, Residual_i represents the residual correction output for the i -th channel, φ_Y and $\varphi_{\hat{Y}}$ denote the mean values of the original and predicted compressed representations, respectively. This latent consistency term encourages the predictor to preserve global statistics (e.g., mean trends) in the compressed space, ensuring more

Table 6: Runtime comparison across models. The reported computational time is measured in seconds

Dataset & Channel		PCDF-MLP	PCDF-Conv	PCDF-Trans	PCDF-Linear	PatchTST	Autoformer	HDMixer	Dlinear	MSGNET	TSMixer
NYC taxi 0.1/24	5	0.00270	0.00152	0.00198	0.00176	0.00393	0.02552	0.01133	<u>0.00047</u>	0.06311	0.00343
	10	0.00302	0.00173	0.00216	0.00192	0.00639	0.02513	0.01993	<u>0.00077</u>	0.07779	0.00529
	20	0.00348	0.00235	0.00281	0.00253	0.01048	0.02604	0.02986	<u>0.00140</u>	0.10720	0.00692
	30	0.00408	0.00301	0.00343	0.00321	0.01514	0.02641	0.03727	<u>0.00197</u>	0.12478	0.00857
	40	0.00465	0.00359	0.00401	0.00376	0.01980	0.02738	0.05849	<u>0.00246</u>	0.19804	0.00923
DC bike 0.3/30	5	0.00260	0.00155	0.00188	0.00165	0.00428	0.02676	0.01103	<u>0.00046</u>	0.05958	0.00324
	10	0.00302	0.00183	0.00208	0.00183	0.00642	0.02688	0.02000	<u>0.00079</u>	0.07809	0.00462
	20	0.00337	0.00234	0.00274	0.00263	0.01083	0.02796	0.02784	<u>0.00135</u>	0.11492	0.00611
	30	0.00403	0.00293	0.00339	0.00312	0.01517	0.02711	0.03737	<u>0.00198</u>	0.11755	0.00748
	40	0.00489	0.00356	0.00471	0.00416	0.02032	0.02742	0.04924	<u>0.00254</u>	0.15987	0.00828
Electricity 0.1/24	5	0.00248	0.00144	0.00186	0.00162	0.00466	0.02887	0.01474	<u>0.00047</u>	0.07086	0.00423
	10	0.00285	0.00175	0.00216	0.00195	0.00700	0.02988	0.02535	<u>0.00085</u>	0.09254	0.00551
	20	0.00370	0.00254	0.00296	0.00305	0.01201	0.03053	0.03563	<u>0.00147</u>	0.14381	0.00724
	30	0.00408	0.00303	0.00343	0.00322	0.01783	0.03135	0.04694	<u>0.00213</u>	0.15304	0.00915
	40	0.00491	0.00383	0.00425	0.00402	0.02387	0.03307	0.08452	<u>0.00278</u>	0.29734	0.01026
Solar energy 0.25/24	5	0.00264	0.00151	0.00194	0.00170	0.00353	0.02359	0.00897	<u>0.00040</u>	0.04768	0.00286
	10	0.00301	0.00185	0.00230	0.00207	0.00521	0.02305	0.01419	<u>0.00071</u>	0.06095	0.00400
	20	0.00344	0.00237	0.00283	0.00254	0.00800	0.02271	0.02110	<u>0.00126</u>	0.08181	0.00512
	30	0.00396	0.00292	0.00334	0.00311	0.01119	0.02281	0.02844	<u>0.00175</u>	0.08308	0.00673
	40	0.00458	0.00363	0.00398	0.00386	0.01436	0.02294	0.03599	<u>0.00215</u>	0.10797	0.00719
Sensor drift 0.1/30	5	0.00259	0.00250	0.00187	0.00163	0.00525	0.03950	0.01678	<u>0.00053</u>	0.08043	0.00456
	10	0.00295	0.00298	0.00211	0.00190	0.00799	0.04868	0.02559	<u>0.00111</u>	0.10722	0.00588
	20	0.00382	0.00413	0.00274	0.00272	0.01465	0.05048	0.04526	<u>0.00155</u>	0.17748	0.00819
	30	0.00433	0.00476	0.00361	0.00342	0.02135	0.04070	0.05797	<u>0.00231</u>	0.16854	0.00913
	40	0.00500	0.00505	0.00398	0.00375	0.02753	0.03732	0.06161	<u>0.00285</u>	0.22166	0.01115
Weather 0.1/25	5	0.00250	0.00138	0.00184	0.00159	0.00482	0.03028	0.01224	<u>0.00050</u>	0.07227	0.00376
	10	0.00283	0.00172	0.00217	0.00192	0.00741	0.03045	0.02204	<u>0.00082</u>	0.09493	0.00493
	20	0.00406	0.00238	0.00281	0.00254	0.01258	0.02922	0.03420	<u>0.00152</u>	0.14586	0.00626
	30	0.00431	0.00324	0.00369	0.00329	0.01878	0.03096	0.04384	<u>0.00210</u>	0.20596	0.00781
	40	0.00464	0.00357	0.00401	0.00383	0.02490	0.03088	0.07657	<u>0.00288</u>	0.28855	0.00934

Algorithm 1 Training Phase of Compression–Prediction Framework

```

1: for epoch = 1 to  $T$  do
2:   Compression:
3:   for  $i = 1$  to  $C$  do
4:     Split  $X^i \in \mathbb{R}^L$  into segments of length  $\tau$ :  $X_{0:\tau}^i, X_{\tau:2\tau}^i, \dots$ 
5:     for each complete segment do
6:        $Y^i \leftarrow \text{Encode}(X_{j\tau:(j+1)\tau}^i, K)$ 
7:     end for
8:   end for
9:    $Y \leftarrow \sum_{i=1}^C Y^i$ 
10:  Prediction:
11:   $\hat{Y} \leftarrow \text{Predict}(Y)$ 
12:  Decompression:
13:   $\tilde{X} \leftarrow \text{Decode}(K, \hat{Y})$ 
14:   $\hat{X} \leftarrow \text{Dense}\left(\frac{\text{Copy}(\tilde{X})}{\sum k^2} + \text{Residual}\left(\frac{\text{Copy}(\tilde{X})}{\sum k^2}\right)\right)$ 
15:  Loss Computation:
16:  Compute  $\mathcal{L}$  as defined in Eq. 31
17:  Parameter Update:
18:   $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$ 
19: end for

```

coherent reconstructions after decompression. The regulation term encourages sparsity and suppresses overfitting.

To improve numerical stability, we use scale-invariant gradient clipping [20]. For each parameter w and its gradient g , we clip the gradient as:

$$g \leftarrow g \cdot \min\left(1, \frac{\alpha \|w\|}{\|g\|}\right), \quad (32)$$

where $\alpha > 0$ is a hyperparameter, g denotes the gradient and w is the corresponding model parameter.

A.5 Result Analysis with MSE

As shown in Table 5, the best scores are underlined and the second-best are bolded. The PCDF variants (MLP, Conv, Trans, and Linear) frequently achieve top results across datasets.

A.6 Result Analysis with Runtime

As shown in Table 6, our variants (PCDF-MLP, PCDF-Conv, PCDF-Trans, PCDF-Linear) consistently achieve the lowest or second-lowest runtime across datasets and channel settings. These results highlight both the accuracy and efficiency of our approach, especially in resource-limited settings.